

LLARS – Script Creator & Framework Guide

Welcome to the LLARS script creator documentation.

LLARS stands for:

Local Library of Automated Routines & Scripts

This guide is written specifically for people creating, converting, maintaining, or contributing scripts for LLARS. It explains how the framework is organized, which template to start from, what belongs in a script, how `Config.ini` communicates with the LLARS GUI, which framework functions should be used, and what must remain intact for a script to work correctly.

The goal is simple: **script creators should write the task-specific automation while LLARS handles the shared framework behavior.**

Table of Contents

1. Before You Start
 2. How LLARS Is Organized
 3. The Script Creator Templates
 4. Choosing the Correct Script Type
 5. Creating a New Script
 6. The Script Editing Markers
 7. RunCount – Basic Loop
 8. RunCount – Advanced Loop
 9. Timer Scripts
 10. MultiTimer Scripts
 11. Color Detection Scripts
 12. The Script Config.ini
 13. Coordinates
 14. Hotkeys
 15. Colors
 16. Optional Features
 17. Sleep and Timer Values
 18. Framework Functions Script Creators Use
 19. Shared Framework Behavior
 20. Framework Presence and Includes
 21. Functions, Labels, and Name Collisions
 22. Logging
 23. Adding a Script to the Script Selector
 24. Testing a New Script
 25. Converting an Older Script to LLARS
 26. Common Script-Creator Mistakes
 27. About LLARS
 28. The Simple Version
-

1. Before You Start

Make sure you have:

- The complete LLARS project folder.
- AutoHotkey **v1.1.37.02**.
- A working RuneScape setup for testing.
- A basic understanding of AutoHotkey v1 syntax.
- A clear idea of whether your script should run by **run count**, **time**, multiple independent timers, or pixel/color detection.

LLARS is an **AutoHotkey v1 framework**. Do not write LLARS scripts using AutoHotkey v2 syntax.

Keep the project folder structure intact while developing and testing.

Start from a LLARS template whenever possible. The templates already contain the framework connection, lifecycle, GUI integration, completion behavior, and required script structure.

2. How LLARS Is Organized

The important creator-facing structure is:

```
LLARS\  
|  
├─ Core\  
|   ├── LLARS.ahk  
|   ├── LLARS_Core.ahk  
|   ├── LLARS_GUI.ahk  
|   ├── LLARS_Labels.ahk  
|   └─ LLARS_Runtime.ahk  
|  
├─ Scripts\  
|   └─ Your Script Folder\  
|       ├── Your Script.ahk  
|       ├── Config.ini  
|       └─ LLARS Logo.ico  
|  
├─ Templates\  
|   ├── RunCount\  
|   |   ├── Basic Loop\  
|   |   └─ Advanced Loop\  
|   ├── Timer\  
|   ├── MultiTimer\  
|   └─ Color\  
|  
├─ LLARS Config.ini  
└─ Script Selector.ahk
```

The **Core** folder contains shared LLARS behavior.

The **Scripts** folder contains runnable scripts.

The **Templates** folder contains starting points for script creators.

The root **LLARS Config.ini** contains LLARS-wide settings. A script's local **Config.ini** contains settings for that specific script.

A normal script creator should **not copy Core code into a script**. Use the framework functions and labels already provided.

3. The Script Creator Templates

LLARS currently provides five script templates:

- **Basic Loop Script Template** – standard RunCount scripts.
- **Advanced Loop Script Template** – RunCount scripts that need different first-run behavior.
- **Timer Script Template** – scripts that run for a user-selected amount of time.
- **MultiTimer Script Template** – timed scripts with multiple independently scheduled actions.
- **Color Detection Script Template** – timed scripts that monitor a pixel/color and react to a state change.

The templates are not end-user scripts. They are development resources.

When creating a script, copy the appropriate template folder into `Scripts`, rename the `.ahk` file and folder as needed, then build the task-specific logic inside the provided editing area.

Do not strip the framework portions out of the template.

4. Choosing the Correct Script Type

Choose the template based on how the automation should operate.

RunCount

Use RunCount when the user should choose **how many complete repetitions** to perform.

Examples:

- Make 100 inventories.
- Complete 50 laps.
- Perform 20 bank trips.
- Repeat one crafting sequence a specified number of times.

Use:

```
LLARS_SCRIPT_TYPE := "RunCount"
```

Timer

Use Timer when the user should choose **how long the script runs**.

Examples:

- Run for 30 minutes.
- Monitor a location for 2 hours.
- Perform an activity until the overall run timer expires.

Use:

```
LLARS_SCRIPT_TYPE := "Timer"
```

MultiTimer

Use MultiTimer when the script is time-based but also needs several independent recurring timers.

Use:

```
LLARS_SCRIPT_TYPE := "MultiTimer"
```

Color Detection

The Color Detection template uses the Timer framework and adds a standard pixel-monitoring pattern.

Use it when a visible pixel/color state controls when an action should occur.

5. Creating a New Script

A good creation process is:

Step 1

Choose the template that matches the script's behavior.

Step 2

Copy the **entire template folder** into the appropriate location under `Scripts`.

Step 3

Rename the folder and `.ahk` file for the new script.

Step 4

Keep the template's framework code intact.

Step 5

Write the task-specific automation inside the script editing markers.

Step 6

Replace the example sections in the local `Config.ini` with the settings your script actually needs.

Step 7

Add clear comments above configuration sections so another person knows what must be selected or configured.

Step 8

Test configuration validation, Start, Pause/Resume, Exit, normal completion, and any script-specific timers.

Step 9

Add the finished script to the Script Selector with its name, setup difficulty, information text, and correct launch path.

6. The Script Editing Markers

LLARS templates contain these exact markers:

```
; =====  
; SCRIPT_EDIT_BEGIN_4C4C415253  
; =====
```

and:

```
; =====  
; SCRIPT_EDIT_END_4C4C415253  
; =====
```

These markers define the task-specific editing region used by LLARS tooling and runtime estimation.

Do not rename, remove, reformat, or duplicate these markers.

For RunCount scripts, the code that represents the repeated work belongs between these markers.

For Timer, MultiTimer, and Color scripts, the operational labels/functions provided for script-specific behavior are inside this region.

7. RunCount – Basic Loop

Use the Basic Loop template when every run follows the same sequence.

The important structure is:

```

Start:

if (!LLARS_StartRun())
    return

Loop, %runcount%
{
    LLARS_BeginLoop()

    ; WRITE NORMAL SCRIPT CODE HERE

    LLARS_EndLoop()
}

LLARS_RunComplete()

return

```

LLARS_StartRun()

This prepares and validates a RunCount session. If startup fails or the user cancels, the script returns without beginning the automation.

LLARS_BeginLoop()

Call this at the beginning of every counted loop.

It allows LLARS to maintain its run state and per-loop behavior.

LLARS_EndLoop()

Call this at the end of every counted loop.

LLARS_RunComplete()

Call this after the RunCount loop finishes normally.

Do not replace the LLARS lifecycle functions with copied legacy GUI, counter, or completion code.

8. RunCount – Advanced Loop

Use Advanced Loop when the first repetition needs different behavior from later repetitions.

The template provides:

```

if (firstrun = 0)
{
    ; WRITE FIRST-RUN CODE HERE

    firstrun := 1
}
else
{
    ; WRITE SUBSEQUENT-RUN CODE HERE
}

```

This is useful when the first run requires:

- A setup or priming action.
- A different starting coordinate.
- A different obstacle or route.
- Opening an interface that remains open afterward.
- Any other one-time action before normal repeated behavior begins.

Code that must happen on **every** repetition should remain outside the `firstrun` condition but inside the main loop.

9. Timer Scripts

Timer scripts use:

```

LLARS_SCRIPT_TYPE := "Timer"

```

The Start label begins with:

```

if (!LLARS_StartTimerRun())
    return

```

The framework creates the timed run and the template starts the countdown.

The Timer template provides a `TimerSetup:` label for one-time startup work and a `ScriptTimer:` label for repeating timer work.

Example:

```

TimerSetup:

SetTimer, ScriptTimer, 1000

return

ScriptTimer:
if (!LLARS_RUNNING)
    return

; REPEATING SCRIPT ACTION

return

```

If you enable an operational timer, make sure it is turned off when the run ends.

The template's `EndMsg` already handles the standard timer completion sequence. Add script-specific timer shutdown where appropriate, but do not remove the standard LLARS completion calls.

10. MultiTimer Scripts

MultiTimer is for scripts that need multiple independent recurring actions during one overall timed session.

The template demonstrates:

```

[Timer One]
option=false
min=10000
max=30000

[Timer Two]
option=false
min=30000
max=60000

```

Each enabled timer can read its own random range and schedule itself.

Example pattern:

```

IniRead, sa1, Config.ini, Timer One, min
IniRead, sa2, Config.ini, Timer One, max
Random, SleepAmount, %sa1%, %sa2%
SetTimer, TimerOne, %SleepAmount%

```

The timer label should normally begin with:


```
if (!LLARS_RUNNING)
    return
```

The template also provides `MultiTimer_StopTimers()`.

When adding another independent timer:

1. Add its `Config.ini` section if configuration is needed.
2. Add its startup scheduling.
3. Add its timer label.
4. Add it to `MultiTimer_StopTimers()`.

Do not forget step 4. A completed run should not leave operational timers running.

11. Color Detection Scripts

The Color Detection template is a Timer script with a standard pixel monitoring pattern.

Its example `Config.ini` contains:

```
[Pixel Coordinate]
type=coordinate
x=
y=

[Target Color]
type=color
color=0xFFFFFF

[Action Location]
type=coordinate
xmin=
xmax=
ymin=
ymax=
```

The template uses:

```
PixelGetColor, currentColor, %x%, %y%, RGB
```

When the current color matches the configured target color, the detection timer is temporarily disabled while the action is handled.

A second timer can wait for the watched pixel to change before re-enabling detection. This prevents the same unchanged state from repeatedly triggering the action.

LLARS uses **Client** coordinates for mouse and pixel operations. Do not add a script-level `CoordMode`, `Mouse`, `Window` or other hardcoded coordinate mode that overrides the framework.

12. The Script Config.ini

Every runnable script should have a local:

```
Config.ini
```

in the same folder as the `.ahk` file.

Typed hotkey, coordinate, and color sections also drive Configuration Status and Reset Config.

Examples include:

- Coordinates.
- Coordinate ranges.
- Hotkeys.
- Colors.
- Sleep ranges.
- Optional features.
- Script-specific timer ranges.

Use meaningful section names.

Good:

```
[Bank Preset]
type=hotkey
hotkey=
```

Less useful:

```
[Key1]
hotkey=
```

Write comments that tell the user **what the setting represents and what they should select**.

For coordinates especially, explain whether the user should select a single pixel, an item area, a button, an object, or another region.

13. Coordinates

To expose a coordinate through the LLARS Configuration button, use:

```
type=coordinate
```

Single coordinate

A single point normally uses:

[Pixel Coordinate]

```
type=coordinate  
x=  
y=
```

Coordinate area

A rectangular/randomized area normally uses:

[Action Location]

```
type=coordinate  
xmin=  
xmax=  
ymin=  
ymax=
```

The script can then choose a random point inside the configured range:

```
IniRead, x1, Config.ini, Action Location, xmin  
IniRead, x2, Config.ini, Action Location, xmax  
IniRead, y1, Config.ini, Action Location, ymin  
IniRead, y2, Config.ini, Action Location, ymax  
  
Random, x, %x1%, %x2%  
Random, y, %y1%, %y2%  
  
NaturalClick(x, y)
```

LLARS owns coordinate mode. Scripts should not override it.

14. Hotkeys

To expose a hotkey through the LLARS Configuration button, use:

[Example Hotkey]

```
type=hotkey  
hotkey=
```

Read it normally:

```
IniRead, key, Config.ini, Example Hotkey, hotkey
```

Then use the value according to the script's needs.

Give the section a name that tells the user exactly which RuneScape action should be assigned.

15. Colors

To expose a color through the LLARS Configuration button, use:

```
[Target Color]
type=color
color=0xFFFFF
```

The framework recognizes color configuration and allows the user to capture it through LLARS.

Read the configured value from the script's local `Config.ini`.

When performing pixel checks, use the same RGB behavior expected by the script and template.

16. Optional Features

An optional feature can use:

```
option=true
```

or:

```
option=false
```

LLARS validation and Configuration Status treat disabled optional sections as not required. Reset Config can still clear saved typed hotkey, coordinate, or color values from those sections.

When your script has an optional feature:

1. Give it a clear `option` setting.
2. Read the option.
3. Normalize it when necessary with `StringLower`.
4. Only execute that feature when enabled.
5. Keep settings belonging exclusively to the disabled feature from becoming unnecessary setup requirements.

Example:

```
IniRead, option, Config.ini, Timer One, option
StringLower, option, option

if (option = "true")
{
    ; ENABLE FEATURE
}
```

17. Sleep and Timer Values

Sleep and timer ranges are stored in milliseconds.

Example:

```
[Sleep Short]
```

```
min=500
```

```
max=1000
```

For a normal RunCount sleep with the enhanced loop estimate:

```
IniRead, sa1, Config.ini, Sleep Short, min
```

```
IniRead, sa2, Config.ini, Sleep Short, max
```

```
Random, SleepAmount, %sa1%, %sa2%
```

```
LLARS_EstimatedSleep(SleepAmount)
```

Use LLARS_EstimatedSleep() for normal configured RunCount sleeps that occur before the end of the loop. These sleeps contribute to LLARS' predictive loop estimate, but they do not trigger the exact final countdown.

On every loop execution path, the actual final randomized sleep must use LLARS_FinalSleep().

Creators may still use the standard AutoHotkey form:

```
Sleep, %SleepAmount%
```

Plain Sleep keeps the original predictive runtime-estimation behavior. The enhanced estimated-sleep system is optional.

For the actual final randomized sleep on the current loop path, use:

```
LLARS_FinalSleep(SleepAmount)
```

Until LLARS_FinalSleep() is reached, Est. Loop Remaining stays predictive. As soon as the final sleep's randomized duration is known, LLARS replaces the estimate with an exact countdown to the next loop. This handoff happens on every loop.

- 1,000 = 1 second.
- 60,000 = 1 minute.
- 300,000 = 5 minutes.
- 600,000 = 10 minutes.
- 3,600,000 = 60 minutes.

Use ranges that make sense for the action. Do not blindly reuse values from an unrelated script.

18. Framework Functions Script Creators Use

These are the most important shared functions for normal script development.

Initialization

```
LLARS_Initialize()
```

Initializes the script's LLARS environment. The templates already call it.

RunCount lifecycle

```
LLARS_StartRun()  
LLARS_BeginLoop()  
LLARS_EndLoop()  
LLARS_RunComplete()
```

Use these for RunCount scripts.

Timer lifecycle

```
LLARS_StartTimerRun()  
LLARS_EndTimerRun()  
LLARS_TimerRemainingText(RemainingTime)
```

Use these for Timer/MultiTimer scripts.

RunCount estimated sleeps

```
LLARS_EstimatedSleep(SleepAmount)  
LLARS_FinalSleep(SleepAmount)
```

Use `LLARS_EstimatedSleep()` for ordinary non-final configured sleeps inside a RunCount loop. Use `LLARS_FinalSleep()` for the actual final randomized sleep on every possible execution path. When `LLARS_FinalSleep()` begins, LLARS switches Est. Loop Remaining from prediction to the exact time until the next loop. Plain Sleep remains supported but does not perform this enhanced handoff.

Clicking

```
NaturalClick(x, y)  
NaturalClick(x, y, "right")
```

Use the shared natural click helper for normal scripted left- and right-clicks instead of creating another copy of the same framework behavior or using raw `Click` / `MouseClicked` commands.

Logging

```
Log("EVENT", "Message")
```

Use the shared logging system for useful script events.

Finishing behavior

```
Logout()
```

The supplied timed templates already use the shared finishing behavior in their completion path.

Use the template as the authoritative example for where these calls belong.

19. Shared Framework Behavior

LLARS Core already handles shared responsibilities such as:

- Finding the LLARS project root.
- Loading the root LLARS Config.ini.
- Loading the script's local Config.ini.
- Startup validation.
- RuneScape window checks.
- Main LLARS GUI creation.
- RunCount GUI behavior.
- Timer GUI behavior.
- Shared LLARS hotkeys.
- Pause/resume state.
- Exit behavior.
- Coordinate, color, and hotkey setup.
- Configuration Status for typed script settings.
- Reset Config for saved typed script settings.
- Logging.
- Natural clicking.
- RunCount lifecycle.
- Timer lifecycle.
- Runtime estimation for RunCount scripts.

Do not recreate these systems inside each script.

The framework exists so the individual script can focus on **what the automation does**, not on rebuilding LLARS.

20. Framework Presence and Includes

Every current template performs a standalone framework-presence check before initialization:

```
if !LLARS_FrameworkAvailable()  
    LLARS_FrameworkError()  
  
LLARS_Initialize()
```

This is intentionally local to the script.

It must work even when someone opens a single `.ahk` directly from a ZIP, RAR, or 7Z archive and the rest of LLARS was not extracted with it.

The template's `LLARS_FrameworkAvailable()` searches upward for:

```
Core\LLARS.ahk  
LLARS Config.ini
```

If the complete project cannot be found, `LLARS_FrameworkError()` explains that the full LLARS folder must be extracted.

The template also contains a series of optional `#Include *i` paths at the **end of the file** so scripts can exist at different folder depths under `Scripts`.

Keep the framework-presence functions and include tail when creating a script.

Do not move a script outside the LLARS project and expect it to operate independently.

21. Functions, Labels, and Name Collisions

This is important when adding helper functions.

AHK functions are global to the combined script after includes are processed.

Therefore, if your script defines a function with the same name as a function in Core, AutoHotkey can report a duplicate function error.

Before adding a general helper function, check whether LLARS already provides one.

Core currently contains shared names including:


```
CheckPOS
CloseOtherLLARS
DisableHotkey
EnableHotkey
DisableButton
EnableButton
LLARS_FindRoot
LLARS_Initialize
LLARS_CheckGame
LLARS_StartRun
LLARS_StartTimerRun
LLARS_EndTimerRun
LLARS_TimerRemainingText
LLARS_ResetRunState
LLARS_BeginLoop
LLARS_EstimatedSleep
LLARS_FinalSleep
LLARS_EndLoop
LLARS_RunComplete
LLARS_CheckStartupFiles
ConfigError
ConfigErrorMessage
CheckConfigFile
LLARS_GetColorKey
GetConfigType
LoggingCheck
StartLogSession
NaturalClick
NaturalNoise
NaturalNoiseLayer
NaturalHash
CalculateScriptRuntime
ParseLLARSRuntime
LLARS_CreateMainGUI
LLARS_CreateRunCountGUI
LLARS_CreateTimerGUI
RandomSleepAmountToMinutesSeconds
LLARS_ShowCoordinatePreview
LLARS_ClientToScreen
LLARS_HideCoordinatePreview
SetLLARSHOTKEYS
```

Script-specific helper names should be descriptive and unique.

For example:

```
Prayer_ReadPotionTimer()
```

is safer than:

```
ReadTimer()
```

The local template functions:

```
LLARS_FrameworkAvailable()  
LLARS_FrameworkError()
```

are intentionally present in runnable scripts and should **not** also be added to Core.

22. Logging

LLARS provides shared logging.

Use:

```
Log("EVENT", "Message")
```

Log events that are useful when diagnosing or understanding a run.

Examples:

```
Log("START", "Starting banking sequence")  
Log("ACTION", "Clicked configured fishing location")  
Log("TIMER", "Potion timer rescheduled")  
Log("COMPLETE", "Script completed normally")
```

Avoid logging every tiny operation unless it is useful. Logs should help a creator or tester understand what happened.

Use the framework's logging system instead of creating a separate script-specific logging framework.

23. Adding a Script to the Script Selector

A finished runnable script should be represented in `Script Selector.ahk`.

The selector needs:

- A visible script name in `scriptArray`.
- An information entry for normal-click details.
- A setup difficulty.
- A concise description.
- Any important prerequisite information.
- A launch mapping to the exact script folder and filename.

Use the existing selector entries as the formatting standard.

Setup difficulty uses:

```
SetSetupDifficulty("Easy")
SetSetupDifficulty("Intermediate")
SetSetupDifficulty("Hard")
```

The information GUI should use the same established sizing as the other entries.

For launch mapping, use `LLARS_SCRIPTS_DIR` rather than rebuilding the root path manually.

Example:

```
Case "Example Script":
    scriptDir := LLARS_SCRIPTS_DIR . "\Example Script"
    scriptFile := "Example Script.ahk"
```

Before finishing, verify that the path actually exists.

24. Testing a New Script

Do not consider a script finished because its main action works once.

Test the complete LLARS lifecycle.

Startup

Confirm:

- The script opens from the complete project.
- Missing framework/project files produce the expected error.
- Missing `Config.ini` is detected.
- Missing required settings are detected.
- Disabled optional settings do not incorrectly block startup.

GUI

Confirm:

- Information opens.
- Required Colors/Coordinates/Hotkeys appear and save correctly.
- Configuration Status reports Ready/Missing/Not Required correctly.
- Reset Config clears only saved typed values and refreshes status.

Start

For RunCount:

- Canceling the run-count prompt does not start.
- Invalid values are rejected.
- A small run count works.
- The count updates correctly.

- ? Confirm normal in-loop delays use LLARS_EstimatedSleep() and the true final delay uses LLARS_FinalSleep().
- ? During every loop, Est. Loop Remaining stays predictive until LLARS_FinalSleep() begins, then switches to the exact randomized remaining time.
- ? Test first-run, subsequent-run, renewal, and other optional branches so each path reaches the correct final sleep.
- ? Normal completion works.

For Timer/MultiTimer/Color:

- Canceling the time prompt does not start.
- Decimal minutes work.
- Countdown updates.
- Operational timers stop when finished.
- Normal completion works.

Controls

Confirm:

- Pause works.
- Resume works.
- Exit always works.
- The script does not continue performing actions while LLARS is paused.

RuneScape behavior

Confirm:

- Coordinates are correct.
- Random coordinate ranges remain inside the intended area.
- Hotkeys trigger the intended action.
- Pixel/color checks use the expected state.
- Script-specific timers reschedule correctly.

Selector

Confirm:

- The script appears in the selector.
- Its information fits the established GUI.
- Its difficulty is appropriate.
- Selecting it launches the correct `.ahk`.

25. Converting an Older Script to LLARS

Older standalone scripts may contain their own copy of systems that LLARS now provides centrally.

When converting one:

Keep

Keep task-specific behavior such as:

- The actual automation sequence.
- Script-specific timers.
- Script-specific pixel/color logic.
- Script-specific helper functions.
- Script-specific configuration.
- Unique operational behavior.

Replace with LLARS

Remove or replace obsolete local copies of:

- Main framework GUI code.
- Shared hotkey management.
- Shared config validation.
- Run-count GUI/counters.
- Timer GUI.
- Generic logging systems.
- Generic natural-click helpers already provided by LLARS.
- Duplicate completion behavior now handled by the selected template.
- Hardcoded framework paths.

Coordinate mode

Do not retain old script-level lines such as:

```
CoordMode, Mouse, Window
```

LLARS Core owns coordinate mode.

Config metadata

Add the appropriate metadata where the GUI needs to expose a setting:

```
type=coordinate  
type=hotkey  
type=color
```

Lifecycle

Move the old task-specific operational code into the appropriate current template rather than trying to preserve an obsolete standalone framework around it.

26. Common Script-Creator Mistakes

Starting without a template

This often leads to missing initialization, completion, validation, includes, or GUI behavior.

Start from the closest template.

Copying Core functions into the script

This can create duplicate function errors and makes the script harder to maintain.

Call shared LLARS functions instead.

Overriding CoordMode

Hardcoded coordinate modes can make saved LLARS coordinates behave incorrectly.

Let Core own coordinate mode.

Using the wrong RunCount sleep helper

Use `LLARS_EstimatedSleep(SleepAmount)` for a normal delay when more loop work follows.

Use `LLARS_FinalSleep(SleepAmount)` for the actual final randomized delay before `LLARS_EndLoop()` on that execution path. Do not leave the final delay as `LLARS_EstimatedSleep()`, or the GUI will remain predictive instead of switching to the exact countdown.

If the final delay changes because of first-run or optional branches, each possible path must call `LLARS_FinalSleep()` at its own true ending sleep.

Using raw click commands for normal scripted clicks

Use:

```
NaturalClick(x, y)
NaturalClick(x, y, "right")
```

instead of raw `Click` / `MouseClicked` commands for normal scripted mouse interaction.

Removing the edit markers

This can break tooling and RunCount runtime estimation.

Keep both exact markers.

Forgetting to stop timers

A timed script may finish its overall run while a script-specific timer remains active.

Turn off every operational timer during completion.

Forgetting LLARS_RUNNING

Recurring operational timer labels should normally protect themselves with:

```
if (!LLARS_RUNNING)
    return
```

This prevents script actions from continuing while the framework says the run is not active.

Poor Config.ini instructions

A section named [Location] with no explanation may be obvious to the creator but not to the user.

Explain what should be selected.

Generic helper function names

Names such as Check(), Reset(), or Timer() are more likely to collide.

Use descriptive script-specific names.

Adding a script but not the selector entry

A runnable script under Scripts should normally be represented in the selector.

Add its information and launch mapping before release.

27. About LLARS

LLARS stands for:

Local Library of Automated Routines & Scripts

The LLARS project, framework, scripts, templates, and documentation are created, maintained, and supported by **Gubna**.

LLARS is free and open source.

GitHub

<https://github.com/Gubna-Tech/RuneScape>

Discord Community

<https://discord.gg/Wmmf65myPG>

Script creators and contributors can use the community to discuss LLARS development, script behavior, setup requirements, testing, and contributions.

28. The Simple Version

If you remember nothing else, remember this:

1. Use AutoHotkey v1.1.37.02.
2. Start from the correct LLARS template.
3. Keep the framework code intact.
4. Write task-specific code inside the script editing markers.
5. Put script-specific settings in the local `Config.ini`.
6. Mark GUI-configurable values with `type=coordinate`, `type=hotkey`, or `type=color`.
7. Use LLARS lifecycle and shared helper functions instead of copying framework code.
8. Do not override LLARS coordinate mode.
9. Check new function names against Core before adding them.
10. Stop every script-specific timer when a timed run ends.
11. Test Start, Pause, Resume, Exit, configuration validation, and normal completion.
12. Add the finished script to the Script Selector with complete information and a verified launch path.

That is the LLARS creator workflow.

The framework handles the shared systems. **The script creator handles the automation.**
